Implementación de PHP 8.2 y Drivers MySQL

Migración de Entorno Estático a Entorno Dinámico

Introducción

Tras validar en la Fase I que la red de AWS y Docker funcionaban correctamente, el siguiente paso lógico ha sido preparar el servidor para ejecutar código real. El servidor Apache que montamos inicialmente solo servía archivos estáticos (.html), pero el Juego de Damas requiere procesar lógica en el backend y persistir datos.

En esta Issue , he actualizado el contenedor para convertirlo en un entorno dinámico basado en PHP 8.2. El objetivo no ha sido solo instalar PHP, sino dejar configurado el "puente" (drivers) necesario para que el servidor web pueda hablarse con la base de datos MySQL de la otra instancia en fases posteriores.

Desarrollo: Justificación de la Migración

El paso a PHP 8.2-Apache

En lugar de instalar PHP sobre el sistema operativo, he mantenido la filosofía de containerización. He sustituido la imagen genérica de Apache por una que integra el motor de PHP. Esto me permite mantener el servidor web aislado y asegurar que la versión de PHP sea exactamente la misma en cualquier entorno de despliegue.

Necesidad de un Dockerfile personalizado

Un problema técnico común es que las imágenes oficiales de PHP son minimalistas y no incluyen los drivers de conexión a bases de datos por defecto. Para que mi código PHP pueda ejecutar consultas en el servidor MySQL (10.1.0.30), he tenido que crear un Dockerfile propio.

Este archivo añade las extensiones mysqli y pdo_mysql, que son los conectores esenciales que actúan como traductores entre el servidor web y la base de datos.

Implementación

Creación del Dockerfile

Cambié la estrategia de usar una imagen "cerrada" a construir una propia. En la ruta /home/ubuntu/mi_proyecto_web/, creé el archivo Dockerfile con las instrucciones para instalar los drivers.

```
ubuntu@ip-10-1-0-85:~/mi_proyecto_web$ nano /home/ubuntu/mi_proyecto_web/Dockerfile
ubuntu@ip-10-1-0-85:~/mi_proyecto_web$ nano /home/ubuntu/mi_proyecto_web/docker-compose.yml
ubuntu@ip-10-1-0-85:~/mi_proyecto_web$ cd /home/ubuntu/mi_proyecto_web
ubuntu@ip-10-1-0-85:~/mi_proyecto_web$ sudo docker-compose down
Stopping mi_proyecto_web_web_1 ... done
Removing mi_proyecto_web_web_1 ... done
Removing network mi_proyecto_web_default
```

Actualización de Docker Compose

Modifiqué el archivo docker-compose.yml para que, en lugar de descargar una imagen de internet, utilice el Dockerfile que acabo de crear mediante la instrucción build: .. También ajusté la ruta del volumen a /var/www/html/, que es el estándar para esta nueva imagen.

Construcción y Despliegue

Ejecuté la reconstrucción del entorno con el comando sudo docker-compose up -d --build. Durante este proceso, Docker compiló las extensiones de base de datos directamente dentro del contenedor.

```
ubuntu@ip-10-1-0-85:~/mi_proyecto_web$ sudo docker-compose up -d --build
Creating network "mi_proyecto_web_default" with the default driver
Building web
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  19.46kB
Step 1/6 : FROM php:8.2-apache
 ---> 4cd3c3572131
Step 2/6 : RUN docker-php-ext-install mysqli && docker-php-ext-enable mysqli
 ---> Running in 7b050af5cb8f
```

Validación del Entorno

Para asegurar que el entorno es sólido antes de subir los archivos del juego, realicé dos comprobaciones críticas:

1. Versión de PHP: Confirmé que el contenedor corre PHP 8.2.30, una versión moderna y segura.
2. Verificación de Drivers: Ejecuté el comando php -m | grep -i pdo dentro del contenedor. Originalmente solo aparecía pdo_sqlite, pero tras la personalización, el sistema confirmó la presencia de pdo_mysql.

```
ubuntu@ip-10-1-0-85:~/mi_proyecto_web$ sudo docker exec mi_proyecto_web_web_1 php -v
PHP 8.2.30 (cli) (built: Apr 22 2026 01:31:20) (NTS)
Copyright (c) The PHP Group
Zend Engine v4.2.30, Copyright (c) Zend Technologies
    with Zend OPcache v8.2.30, Copyright (c), by Zend Technologies
```

```
ubuntu@ip-10-1-0-85:~$ sudo docker exec mi_proyecto_web_web_1 php -m | grep -i pdo
PDO
pdo_mysql
pdo_sqlite
```

Conclusiones

Con la Issue finalizada, el servidor ha dejado de ser un simple visor de archivos para convertirse en una plataforma de ejecución dinámica.

Ahora que tengo los drivers de base de datos cargados y el motor PHP 8.2 operativo, los "puentes" están construidos. El sistema está listo para recibir los archivos del juego (api.php, GameLogic.php, etc.) y empezar la comunicación real con el servidor de base de datos MySQL en la red privada de AWS.